SOFTWARE ENGINEERING LAB – EXPERIMENT 5

UML Class Diagram – Project-Based Student Template

Group No.	1
Project Title	KV Store – Distributed Transactional Key-Value Store
Student Name(s) / Roll No.	Aditi Garg (20251651008), Neha Nupur (20251651064), Shivam Kumar Soni (20251651084)

Aim

To identify the major classes of our project and prepare a UML class diagram with attributes, methods, relationships and multiplicities.

Step 1 – Write a Short Description of Your Project

In 2–4 simple sentences, write what your system does:

kvstore is a distributed, replicated, transactional key-value store that lets client applications read, write, delete, and transactionally update string keys mapped to byte-sequence values.

It uses Raft-based consensus for fault-tolerant replication across cluster nodes, multi-version concurrency control (MVCC) for snapshot-isolated transactions, and a checksummed write-ahead log for crash durability.

A sharding layer (rendezvous hashing) together with a two-phase commit coordinator lets the system scale horizontally across multiple independent Raft clusters with atomic cross-shard writes.

An integrated operational intelligence layer continuously monitors cluster health and surfaces anomalies, trend forecasts, and leadership recommendations through a REST API and live dashboard.

Step 2 – Identify Candidate Classes

Look at your project title and requirements. Write important nouns/entities. Keep only those that need their own data or behaviour.

Possible Class	Why is it needed?	Keep? (Yes/No)
RaftCluster	Represents the replicated group of nodes and tracks the current leader	Yes
ClusterNode	Each replica has its own role, term, and log — needs independent state	Yes
HybridLogicalClock	Produces the causal timestamps every write and version depends on	Yes

Possible Class	Why is it needed?	Keep? (Yes/No)
RaftLog	Ordered log of entries a node must persist and replay	Yes
LogEntry	One record in the Raft log, with its own checksum and term	Yes
WriteAheadLog	Durability primitive every node writes through before acknowledging	Yes
MVCCVersionStore	Holds the versioned key data used for snapshot isolation	Yes
KeyValueVersion	One timestamped version of a key — has its own visibility rule	Yes
Transaction	Represents a client's multi-key read/write unit and its status	Yes
CrossShardTransaction	Specialised transaction that spans more than one shard	Yes
MetricsCollector	Collects latency/throughput data independently of consensus logic	Yes
OperationalIntelligenceEngine	Detects anomalies and forecasts trends from collected metrics	Yes
TLSConfig	Encapsulates certificate/key configuration for encrypted connections	Yes
ClientLibrary	Discovers the leader and retries requests on behalf of a client	Yes
HttpApiLayer	Exposes REST endpoints and the live dashboard to external callers	Yes
ShardGateway	Routes keys to the correct independent shard cluster	Yes
TwoPhaseCommitCoordinator	Coordinates atomic commit/abort across shards for a cross-shard txn	Yes
StatsProvider (interface)	Narrow read-only interface the AI engine needs — kept as an interface, not a concrete class	Yes
Executor (interface)	Narrow read-only interface the AI engine needs — kept as an interface, not a concrete class	Yes
NodeAddress	Just a string field on ClientLibrary/ClusterNode — no independent behaviour	No
Timestamp	Plain value type returned by HybridLogicalClock — not a class of its own	No

Step 3 – Add Attributes and Methods

For each final class, write only the important data (attributes) and actions (methods).

Class Name	Attributes (2–5)	Methods / Operations (1–4)
RaftCluster	members: ClusterNode[] leaderID: string	electLeader()
ShardGateway	shardList: RaftCluster[]	routeKey(key): RaftCluster
StatsProvider «interface»	— (interface, no attributes)	getStats(): Metrics
Executor «interface»	— (interface, no attributes)	execute(operation): Result

Class Name	Attributes (2–5)	Methods / Operations (1–4)
ClusterNode	role: NodeRole currentTerm: int	startElection() becomeLeader()
TwoPhaseCommitCoordinator	decisionLog: Map<txnID,Decision>	prepare(txn) commitOrAbort(txn)
HttpApiLayer	routes: RouteTable	handleRequest(req)
ClientLibrary	currentLeaderAddress: string	get(key) put(key, value)
HybridLogicalClock	physicalTime: int64	now(): Timestamp
RaftLog	entries: LogEntry[]	appendEntry(e)
WriteAheadLog	filePath: string	appendRecord(data) flush()
MVCCVersionStore	versionChains: Map<key,List>	readVersion(key, ts) writeVersion(key, value, ts)
MetricsCollector	histograms: Map<string,Histogram>	recordLatency(op, ms)
OperationalIntelligenceEngine	anomalyThreshold: float	detectAnomaly(metric) forecastTrend(series)
TLSConfig	mutualTLSEnabled: boolean	loadCertificate()
LogEntry	term: int checksum: string	verifyChecksum()
KeyValueVersion	timestamp: HLCTimestamp deleted: boolean	isVisible(readTs): boolean
Transaction	beginTimestamp: HLCTimestamp status: TxnStatus	begin() commit()
CrossShardTransaction	decisionRecorded: boolean	prepareAllShards()

Step 4 – Decide Relationships and Multiplicity

Class A	Relationship	Class B	Multiplicity at A	Multiplicity at B
RaftCluster	Composition	ClusterNode	1	1..*
ClusterNode	Composition	HybridLogicalClock	1	1
ClusterNode	Composition	RaftLog	1	1
RaftLog	Composition	LogEntry	1	0..*
ClusterNode	Composition	WriteAheadLog	1	1
ClusterNode	Composition	MVCCVersionStore	1	1
MVCCVersionStore	Composition	KeyValueVersion	1	0..*
MVCCVersionStore	Association	Transaction	1	0..*
Transaction	Association	KeyValueVersion	1..*	1

Class A	Relationship	Class B	Multiplicity at A	Multiplicity at B
CrossShardTransaction	Inheritance	Transaction	—	—
ClusterNode	Composition	MetricsCollector	1	1
ClusterNode	Aggregation	OperationalIntelligenceEngine	1	0..1
ClusterNode	Aggregation	TLSConfig	1	0..1
ClientLibrary	Association	ClusterNode	1	1..*
ShardGateway	Association	RaftCluster	1	1..*
ShardGateway	Composition	TwoPhaseCommitCoordinator	1	0..1
TwoPhaseCommitCoordinator	Association	CrossShardTransaction	1	0..*
ClusterNode	Aggregation	HttpApiLayer	1	0..1
HttpApiLayer	Realization	StatsProvider	—	—
HttpApiLayer	Realization	Executor	—	—
OperationalIntelligenceEngine	Dependency (“uses”)	StatsProvider	—	—
OperationalIntelligenceEngine	Dependency (“uses”)	Executor	—	—

*Multiplicity reminder: 1 = exactly one, 0..1 = zero or one, * = many, 1..* = one or many.*

Step 5 – Draw the Final UML Class Diagram

Use the space below OR draw it using draw.io / StarUML / any UML tool. Each class should contain: Class Name | Attributes | Methods.

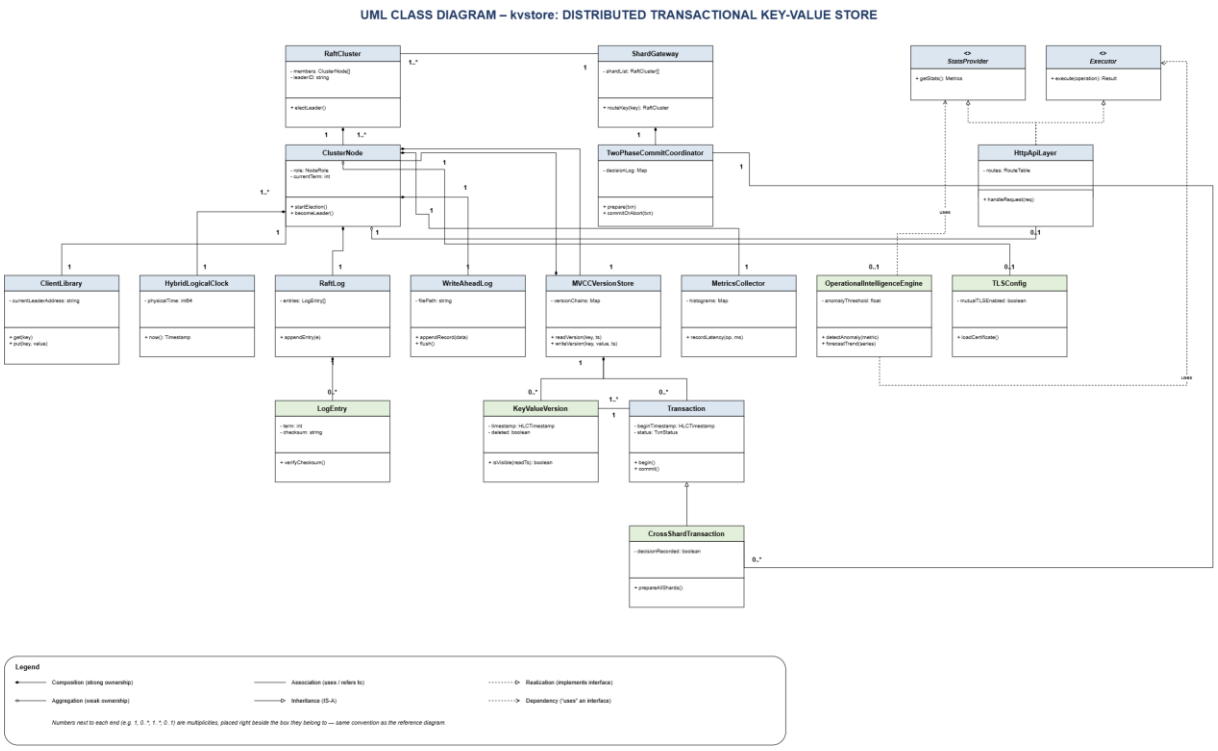


Figure 1: UML Class Diagram – kvstore (Distributed Transactional Key-Value Store)

Quick Self-Check

- ☑ My diagram matches my project title and requirements.
- ☑ I selected only meaningful classes.
- ☑ Important classes have attributes and methods.
- ☑ Relationships are shown clearly.
- ☑ Multiplicity is written where needed.
- ☑ Inheritance is used only for a true IS-A relationship.
- ☑ The final diagram is readable and I can explain my choices.

Result

In this experiment, we identified the major classes for the project titled “KV Store – Distributed Transactional Key-Value Store” and prepared a UML class diagram showing their attributes, methods, relationships and multiplicities.